Uncovering Football Insights: Advanced Match Analytics with Cypher

Course: ST207

Canidate Numbers: 49150, 39706, 45417

Page Count: 10

Summative 3 Report: Football Match Insights Database

ii: Description of Chosen Topic

We chose **football**, deriving insights beyond basic queries, instead using cypher to perform detailed investigations into variables that influence match results. This topic was chosen because it allowed for a combination of detailed statistical analysis and practical use cases, such as the quantitative impact of home advantage and created a "dream team" based on player performance metrics. Additionally, we examined individual player contributions and evaluated how team win rates change when key players are unavailable due to injuries. The database relied on **structured data** scraped from fbref, a platform providing detailed match and player statistics. Moreover, we included PNG images of players to support visualizations of complex query results. This challenged us support our complex queries through visualisations, providing a comprehensive and interactive tool for football analytics.

iii: Description of Data

DATA SOURCE: For our project, we scraped publicly available data from FBref, which includes a mass amount of real world data, enabling us to create 3042 nodes, with 49,884 relations. Using FBref, we scraped in detail statistics on match, team, player, player performance in each match and venue. This is the link: https://fbref.com/en/

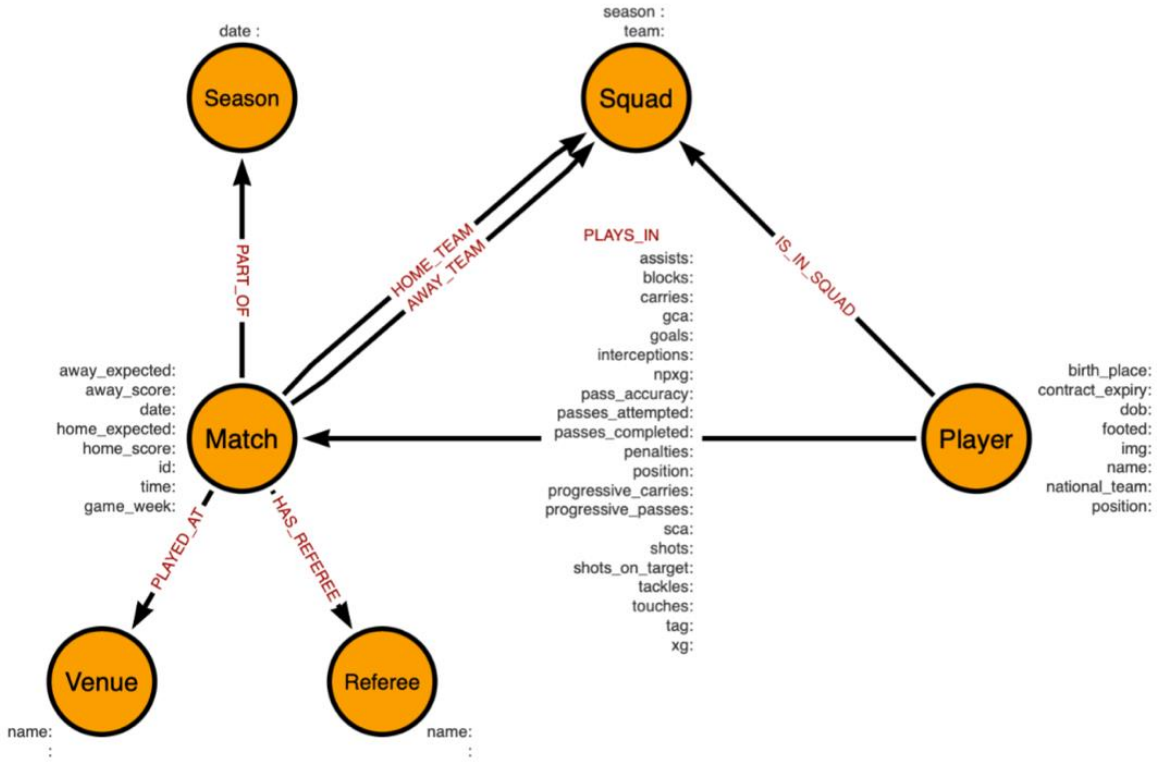
QUALITY OF DATA: The data we gathered from FBref is highly reliable and free from bias in reporting, as it is non subjective and quantitative data. The data is complete, as we observed no missing values, apart from match scores for future games, as expected. The data was mostly well-structured, though we did encounter some challenges with player details, which required additional cleaning. This was due to the fact the data wasn't tabular and was all amongst paragraphs on the players info page. To handle this, we had to iterate through each text box on the html page and extract the relevant details. Some players didn't have complete information such as their contract expiry date – so the amount of information available differs from player to player.

CONCERNS: Our biggest concern with this project was with gaining access to such data, as it is often commercially restricted. This is because the data can be used for betting markets and can help people ascertain better odds. This made sourcing free and reliable data hard as many api providers wanted to charge for current/historic data, or only gave access to top-level statistics which weren't very informative. This is what lead us to do web scraping for the data instead. Given the open and reliable nature of FBref, the main concern we encountered was the rate limits imposed on the website, where requests were timed out after 20 requests per minute. To overcome this, we used a rate limiter to keep track of how many requests are made in a minute and then randomised the wait time between requests. This helped the web scraping tool to not be detected as a bot, which is also causes a connection timeout.

iv: Description of Database Model

Conceptual Description and Property Graph of Neo4J Model

JUSTIFICATION: Initially, we considered using a Team node that would store only the team name and be linked to multiple Squad nodes, where each squad was part of a team. However, we realized this structure would be redundant and inefficient, as it would require an additional node for each team without adding meaningful value to the model. Instead, we decided to store the team name directly within the Squad node as a property. Moreover, storing the team name in the Squad node ensures that we can easily track team compositions and changes across seasons. Since the squad composition changes significantly each season (see graph on pg. 5), there is no need to have a separate Team node that remains static across different seasons.



Entities, Attributes and Relationships

SQUAD: It should be noted that this database stores a new Squadfor each season, because squad line ups change significantly over seasons due to transfers and loans. Therefore, by creating a new Squad node per season, we can track squad compositions and assess changes in teams over time. Each squad has the attributes team and season.

SEASON: Each season has the property date, which scores the dates each season occurs. This is useful for determining what match was in each season (based on game week). Season is important for gaining insight into changes over time, such league tables and team growth.

MATCH: Each match holds a number of useful attributes, including away score/expected score, home score/expected score, date, game week, time and id. The home score and away score can be linked to teams depending on the HOME_TEAM and AWAY_TEAM relationship.

PLAYER: This entity stores the attributes of each player, specifically their: birth place, date of birth, name, left/right footed, national team, position and a link to an image of them. This information is useful for visualisation queries. The position is determined by the majority played position. The nodes for each player are unique, even if a player has played for multiple clubs.

REFEREE: This entity stores the attribute of the referee's name. Each referee is represented as a node in the database and is linked to matches they have officiated.

VENUE: This stores which venue name the match was played at, which is useful for filtering on specific venues and exploring stadium specific trends.

PART_OF: Each match is part of one season and each season can have multiple matches. This can be used to filter matches by season.

PLAYS_IN: A player plays one or more matches and matches have multiple players. This relationship holds many properties and can be used to find specific data on a player in a match, such as touches,

HOME_TEAM / AWAY_TEAM: Each match must have a home and away team, which is the squad that are playing at home or away respectively. This relationship connects match and squad depending on where the match is being played.

PLAYED_AT: This connects MATCH to the venue in which is the match is played. Each match must have exactly one venue.

HAS_REFEREE: This connects referee nodes to the matches that they referee. Each match must have at least of referee, and referees can referee multiple matches.

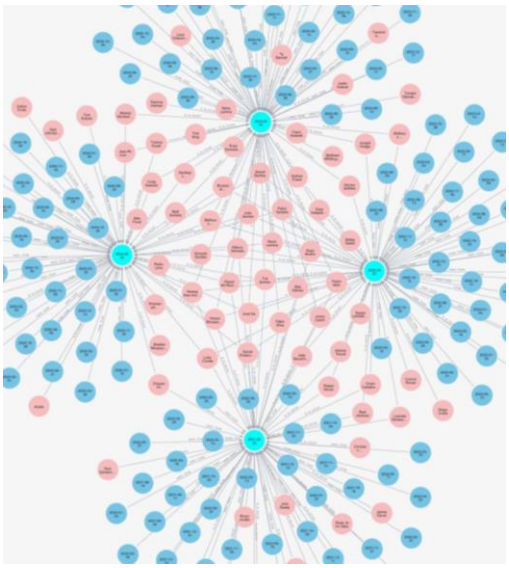
IS_IN_SQUAD: This connects the player node to the squad that they play in. If player is transferred there will be multiple IS_IN_SQUAD relationships for this player.

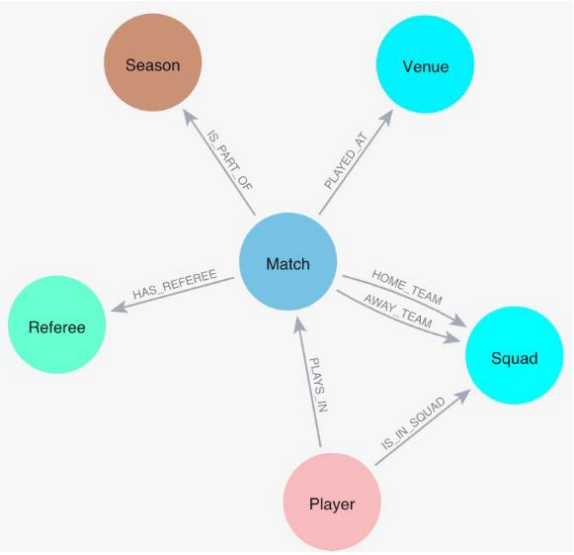
Restrictions and Data Types

A restriction of this data model is how that it fails to account for transfers within the season. The business rules for football are that most transfers happen during the summer transfer window, which is between seasons. Each Squad node represents a team in a specific season, meaning players are linked to squads before the season starts. This structure assumes all transfers happen between seasons, which is mostly true but fails to model January transfers accurately.

The data stored within the properties has been explicitly defined with specific data types to ensure accurate queries. For instance, match scores are stored as integers since they must be whole numbers, while expected scores are stored as floats because they can have decimal values, as seen on FBref. It's also important to note that if a game has not yet been played, the scores are stored as **NULL** values. This is significant because Cypher allows you to filter effectively by NULL values, whereas FBref originally used "N/A" to represent missing data.

Instance Graph Example: The graph on the left highlights how vastly squads change over seasons and therefore the need for seperate squad nodes per season and why a singular team node would not be suitable.





v: Textual Description of Queries

Query 1: Impact of Player Injury

This following query analyses the impact of an individual players injury on the teams success. It does this by comparing the win ratio of the players teams matches when the player is and is not playing. This gives us insight into how impactful a player is on the teams success. It should be noted that there may be other factors that effect the win rates, such as weather and opposition team. This query focuses on the season 2022-2023 and only analyses players who played over 3 matches as some players may just be very short term loans.

STEP 1: Creating a list of names of players who played over 3 matches in the 2022-2023 season

The first step to this query MATCHES players to their squad and the matches they have played it.Next, it usees WHERE to filter for the season of interest, 2022-2023, using the season property stored in squad. Then it counts the DISTINCT matches the player has played and uses WHERE to filter these to players that have only played more than 3 matches.

STEP 2: Calculating the win ratio for the games the player DID play

This query loops through the names stored in player_names_min_games to extract the win ratio for each of these players. First, it uses MATCH to find the players squad, and the matches the squad has played (both home and away), and then matches this to the matches that the player played in. It also matches match to the season that it is played in. Next, it uses WHERE to filter for the season of interest, 2022-2023, using the season property stored in squad and the season node. The WITH is used to aggregate the number of matches played by the player. The SUM then counts how many of these matches played are wins (draw is counted as a loss). If the player's squad is the home team and the home score is greater than the away score, or if the player's squad is the away team and the away score is greater than the home score, it's considered a win. It then RETURNS the name of the player, the total games they played their win ratio and if no games were played (total_games = 0), the win ratio is set to 0%.

STEP 3: Calculating the win ratio for the games the player DID NOT play

This does the exact same steps as STEP 2, however it filters for the matches that the player did not play and their squad still did, using 'AND NOT (player)-[:PLAYS_IN]->(match)'.

STEP 4: Creating Win Ratio Dataframe

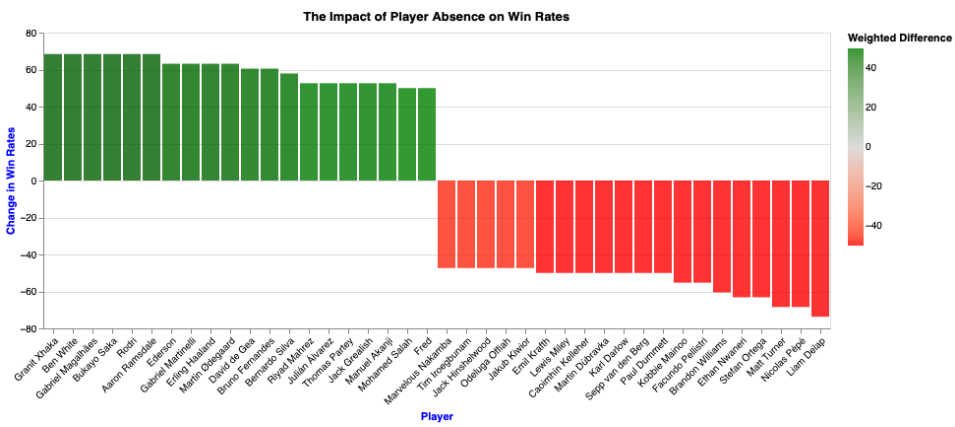
This uses the player_stats list of dictionaries to create a dataframe needed for visualisations. **NOTE** that in order to calculate the impact of their absense a WEIGHTED difference was performed. The ratios are weighted by the number of games they did/did not play. This is because if a player only missed one match, but the team won, the not playing win ratio would be 100%.

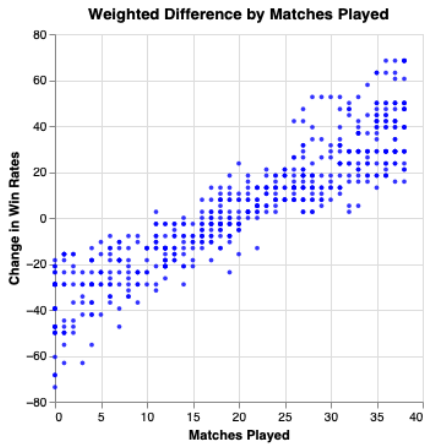
OUTPUT

This is a screenshot of the win ratio data frame created by this query; it has been stored as a CSV for readability.

Player	Playing Win Ratio (%)	Not Playing Win Ratio (%)	Matches Played	Games Missed	Weighted Difference
James Tarkowski	21.052631578947400	0.0	38	0	21.052631578947400
Tyrick Mitchell	27.77777777777780	50.0	36	2	23.68421052631580
Jordan Pickford	18.91891891891890	100.0	37	1	15.789473684210500
Declan Rice	29.72972972972970	0.0	37	1	28.94736842105260
Ollie Watkins	45.94594594594600	100.0	37	1	42.10526315789470
Gabriel Magalhães	68.42105263157900	0.0	38	0	68.42105263157900
Bruno Fernandes	62.16216216216220	0.0	37	1	60.526315789473700
Tomáš Souček	30.555555555555600	0.0	36	2	28.94736842105260
Emiliano Martínez	47.22222222222220	50.0	36	2	42.10526315789470

VISUALISATIONS: The following visualisations use the win ratio data frame created with the query and the package 'Vega Altair' to gain further information into the impact of injured players on win rates. Below is a bar chart of the top and bottom 20 players based on their change in win rates when they are not playing. The win rates data frame has 545 players which is too many to effectively plot, thus this shows the players whose absence had the biggest and least impact on their teams win rate.





The scatter graph plots the relationship between matches played by a player (X) and their change in win rates when not playing for the team (Y). It shows a strong positive correlation between matches played and win rates. This suggests that better players (that have high change in win rates) play more matches. This supports the logic that if a player is better and thus more impactful, they will be chose to play more matches.

Query 2,3 and 4: Home Advantage Analysis

This section performs 3 queries to investigate how teams playing at their home grounds improves their win rates.

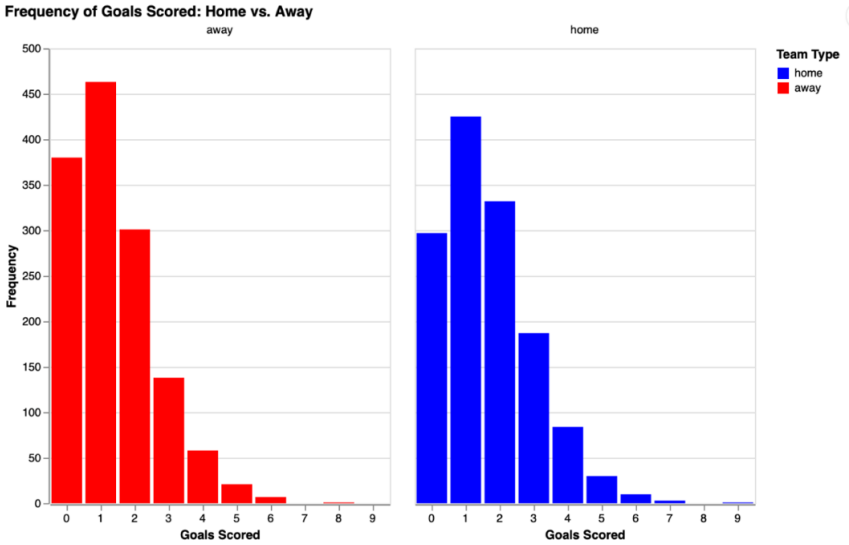
Query 2: Distribution of Frequency of Goals in Home vs Away

PURPOSE: The purpose of this query is to investigate how many times a specific number of goals was scored by home teams across all the matches. Specifically, how this differs for home and away teams and if home teams have a goal distribution that is positively skewed due to their home advantage.

team_type	goals_scored	frequency
home	0	297
home	1	425
home	2	332
home	3	187
home	4	84
home	5	30
home	6	10
home	7	3
home	9	1
away	0	380
away	1	463
away	2	301
away	3	138
away	4	58
away	5	21
away	6	7
away	8	1

CYPHER STEPS: Matches on match, and aliases it to m, then filters it for cases where home score is not NULL, meaning that match has been played. WITH passes the needed data to the next part of the query, aliasing both team type and score. It then RETURNS the frequency each time home score, this is how many times a specific number of goals was scored by home teams across all the matches. The data returned is then stored into a data frame (goal_distribution_df) which is used for visualisations. The data frame/(exported at csv for readability) it the output of this cyber query.

VISUALISATION: This visualisation uses Vega Altair and plots the information extracted above from the cypher query. From this graph is is clear that goals scored by home teams are slightly greater than away teams. However, it is not clear the extent to which playing at home has an advantage, Thus, the next query quantifies the advantage.



Query 3: Average Number of Goal for Home and Away

PURPOSE: This returns the average goal difference for matches, categorized by match result (home win, away win, or draw). The output indicates how much home/away teams win/loose by on average across all seasons (2021-2025)

CYPHER STEPS: First, it Matches on match, and aliases it to m, then filters it for cases where home/away score is not NULL, meaning that match has been played The WITH then passes the home and away score forward and aliases them. The CASE is used to calculate the match result: if the home team's score is greater then the away team's score then result is 'home', if the away team's score is greater then the result is 'away' else the result is 'draw'. The RETURN then uses AVG to calculate the average goal difference across all the matches.

OUTPUT: The output indicates that home teams, on average, **win by a margin of 1.974 goals**, whereas away teams **lose by 1.922 goals**. This suggests that home teams have a significant advantage over away teams.

```
Average goal difference for home team is 1.9738134206219307
Average goal difference for away team is -1.9227272727272728
```

Query 4: Which Teams Have the Greatest Home Advantage

PURPOSE: This query matches teams that have played each other twice during the 2022-2023 season: once at team 1's home stadium and once at team 2's home stadium. By matching on the same teams under different conditions (one as the home team and the other as the away team), it allows for a more accurate comparison of performance in both teams' home stadiums. It then gather data on the home stadiums capacity using the VENUE node to investigate if there is a relationship between capacity and home advantage.

STEPS: Firstly, it MATCHES all the match nodes that are the season 2022-2023 and then matches all squads to matches they played in this season using the home/away team relationships. The CASE clause the determines if the team is playing at home or away based on the relationship HOME_TEAM. It then stores this as match type. The second CASE then stores 'score for', which either the home or away score depending on if they are playing at home or away. The THIRD CASE assigns the opposing team's score based on whether the team is playing at home or away. Next, the WITH aggregates the score_for and score_against values across all matches for each team, and groups these by team_name and match_type This calculates the total goals scored by the team when playing a type of match (home or away). Next, the WITH calculates the home advantage using the total_score_for and total_score_with that were calculated in the previous WITH: For matches where the team played at home, it calculates the total score difference goals scored minus goals conceded). It then does the same for matches where the team played away. It then

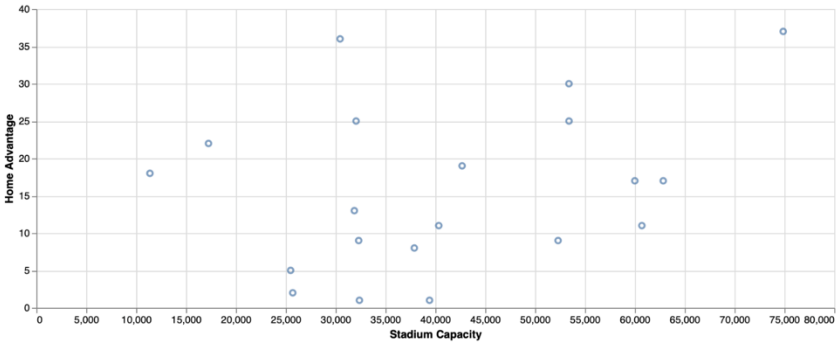
returns the relevent data and orders it by net home advantage, which is the difference between a teams home advantage and away disadvantage. This data is then stored in a data frame. The following formulas were used to calculate the home advantage:

$$\textit{Net Home Advantage} = (\textit{Total Goals Scored at Home} - \textit{Total Goals Conceded at Home}) - (\textit{Total Goals Scored Away} - \textit{Total Goals Conceded Away})$$

Next, a second query is ran that gathers data about the capacity of each venue. First, it finds all Match nodes that are connected to a Venue node via the PLAYED_AT relationship. Then it finds the home teams for each match. It then RETURNS a distinct list of teams and their stadium capacities (stadium_capacity). Logically, the venue the match is played at is the home teams stadium. DISTINCT ensures that each team appears only once, preventing duplicate entries.

OUTPUT: This is a screenshot of the start of the data frame which is the output from the two queries. At face value, it appears as though there may be some relationship between home advantage and capacity larger teams have a greater home advantage. However, the visualisation created below suggests that this seems to be uncorrelated

	team	home_advantage	stadium_capacity
0	Manchester Utd	37	74879
1	Nott'ham Forest	36	30445
2	Liverpool	30	53394
3	Wolves	25	32050
4	Manchester City	25	53400
5	Brentford	22	17250
6	Aston Villa	19	42682
7	Bournemouth	18	11379

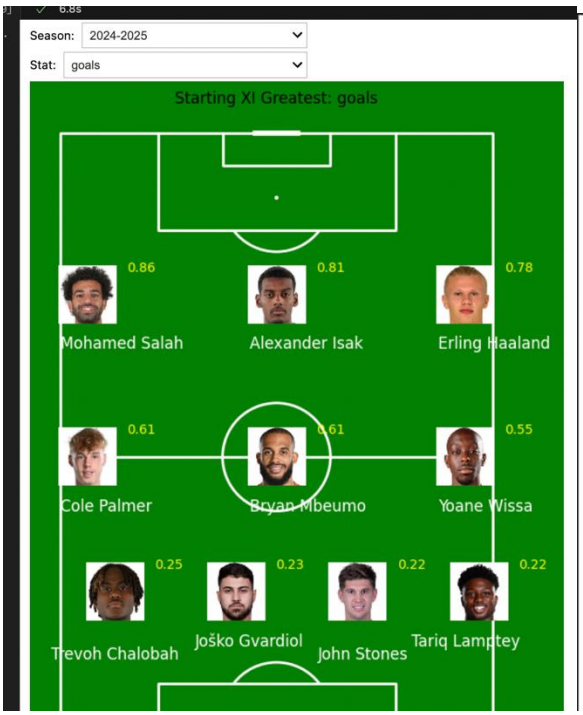


Query 5: Best Team By Stat

PURPOSE: This is a generic query to retrieve a fantasy team (in a 4-3-3 formation) with the highest selected stat for a particular season (excluding goalkeepers). Example query: fantasy team with most goals in 2021-2022 season.

CYPHER STEPS: The cypher query filters out players with no image in order to ensure every player in team has a photo. Then, the query calculates player averages for desired statistic in desired season using cypher's AVG() function. Next, the query orders defender, midfielders and forwards separately based on the average of the statistic. Then, the function excluded_players is used within query to ensure player that play multiple positions are not repeated. Note that the defenders limited to 4, while midfielders and forwards limited to 3, returning a 4-3-3 team

INPUT/OUTPUT : User selects desired statistic and season from drop down menu in widget. Then get_top_team_by_stat() function is called with season and query_stat parameters. This function creates a cypher query as an f string, including user's selected statistic and season. This cypher query is then run using driver. Players stored in 3 separate dictionaries, defenders, midfielders and forwards. These dictionaries are used as parameters for the visualise_team() method. This function is ran in order to output a football pitch with player faces, using mplsoccer library.



Query 6: Get Best Overall Possible Team For Given Season

PURPOSE: This query aims to retrieve a fantasy starting XI with the best overall statistics for a selects season. In order to do this, scores out of 10 are calculated for players from weighted metrics such as goals scored. Different positions have their stats weighted differently with defenders, for example, having their average tackles per game stat weighted higher than average shots. The weightings used are below. The scoring system awards the player with the best overall stats for their position (fw, mid or df) a 10, and the player with the worst overall stats for their position a 0 with score distributed between accordingly. The 4 best defenders, 3 best midfielders 3 top forwards are output to user using mplsoccer and matplotlib for visualisation.

Stat	Defender Weighting	Midfielder Weighting	Forward Weighting
Blocks	0.2	0	0
Interceptions	0.2	0.1	0
Tackles	0.2	0.1	0
Goals	0.05	0.3	0.5
Assists	0.05	0.1	0.1
xg	0.05	0.1	0.175
Progressive Carries	0.05	0.1	0.05
Progressive Passes	0.2	0.1	0.025
Shots	0	0.1	0.15

CYPHER QUERY STEPS:

Highest Stat Query: This query is used to calculate the highest stat recorded for any player within a given season, to be used for later normalisation. First, it matches players to match to season, ensuring correct season, collecting player stats through PLAYS_IN relationship. Next, it filters out any nodes with null value for selected stat. Then it calculates average of stat using AVG() and then ORDER by the calculated average. Lastly, it sets Limit to 1, and return player with highest average stat

Get Best Team Query: This is the query that retrieves the top team from the database. Cypher query begins with matching defenders that have played in the desired season. Averages are calculated for blocks, interceptions, tackles, goals, assists, xg, carries and progressive passes. The only statistic not included here that is in others is shots. This is because while defenders should still be awarded for scoring goals, we do not expect defenders, even of the highest level, to be taking a significant number of shots per game. Each of the players average stats are then divided by the corresponding maximum average of that statistic i.e avg_blocks / {max_blocks}. This is a process of normalisation and ensures stats that are typically higher, e.g progressive passes, are not represented more than stats that are typically lower, e.g goals. On top of the normalisation process, individual stat averages are weighted so that stats typically associated with defence are more significant in deciding final score.

$$NormalisedAttributeScore_{ij} = W_j \cdot \frac{AVG(attribute_{ij})}{MAX(attribute_j)}$$

The Raw performance score for each player is calculated by summing these attributes

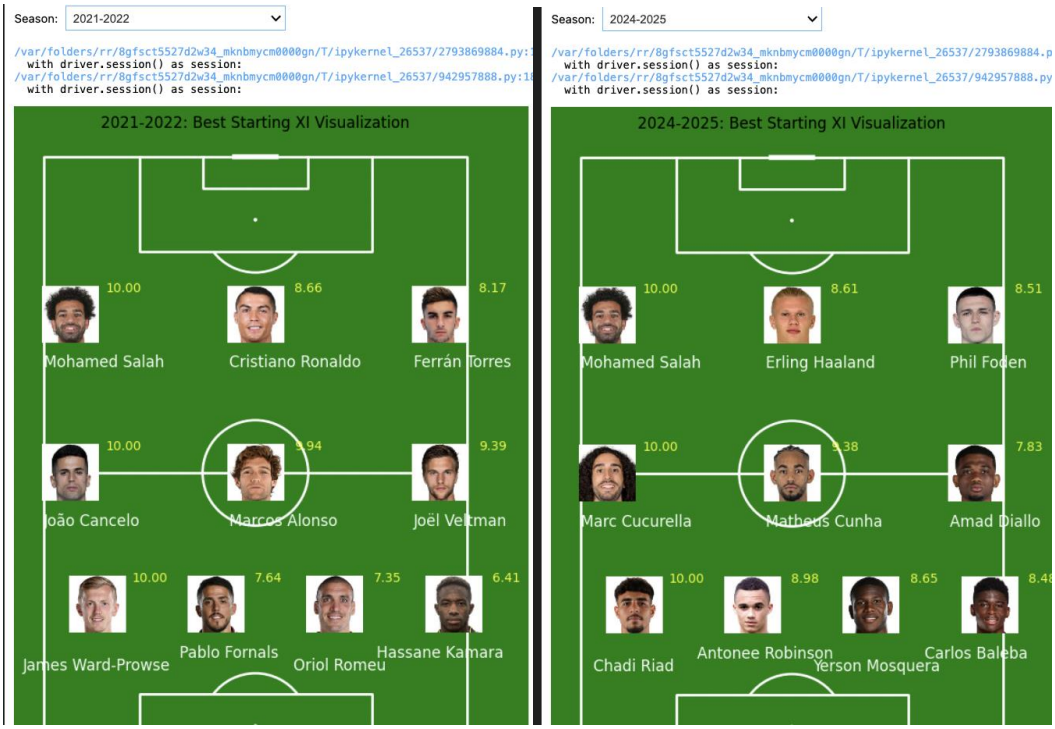
$$RawPeformanceScore_i \ = \sum_j NormalisedAttributeScore_{ij}$$

The maximum and minimum raw performance scores for defenders are also stored for later normalisation. This process is repeated for midfielders and forwards using different weights and selected stats to contribute to score (weighting in table below) Once raw performance scores have been calculated for all positions we can begin final normalisation process using the formula:

$$Final\ Normalised\ Score = 10 \cdot \frac{RawPerformanceScore_i - MIN(RawPerformanceScore)}{Max(RawPerformanceScore) - MIN(RawPerformanceScore)}$$

Once positions are given normalised scores they can be limited accordingly to return a 4-3-3 formation. I.e limit 4 defenders and limit 3 midfielders and forwards.

INPUT/ OUTPUT: User selects desired season from drop down menu in widget. Then, the Get_best_team() function is called with selected season as a parameter. This function first calculates the maximum value for the chosen season of all the different stats being measured. In order to do this, the find_highest_stat() function is called, loading in the necessary statistic and the user's selected season.. These max_stats are loaded into the query alongside the chosen season and returns three dictionaries: defenders, midfielders and forwards, these are used to call the function visualise_best_team(). This function uses mplsoccer to visualise a football pitch, before placing player pictures and stats on pitch according to whether they are midfielders forwards or defenders.



Query 6: Defender with Most Growth Between any Two Season

PURPOSE: This query aims to retrieve a list of defenders that had the greatest season on season improvement in their raw performance score (defined above).

CYPHERSTEPS: In order to normalise stats to calculate a raw performance score, again the max of each stat needs to be calculated. However, this time rather than finding the highest average within a given season, we are finding the highest average over all games played for each statistic. This is to give a consistent measurement of a player's ability between seasons to ensure comparibility. Since if, for example, all players scored more goals in a particular season, this would compromise the measurement of Indvidual player growth regarding goals since they would be less significant the following year. These max stats are calculated using the find_highest_stat_oat() which works similarly to find_highest_stat() but without matching players and performances to a specific season. The cypher query begins by defining a list of all seasons to loop through each subsequent season. Season1 and season2 are used to store two consecutive seasons, season1 being the prior season. Defenders and performances that are a part of both

seasons are selected. Averages for every relevant stat is calculated both for season 1 and season 2. Each of these stats is then normalised using the max stats calculate prior to the query and also weighted with same weights as in the get_best_team() query

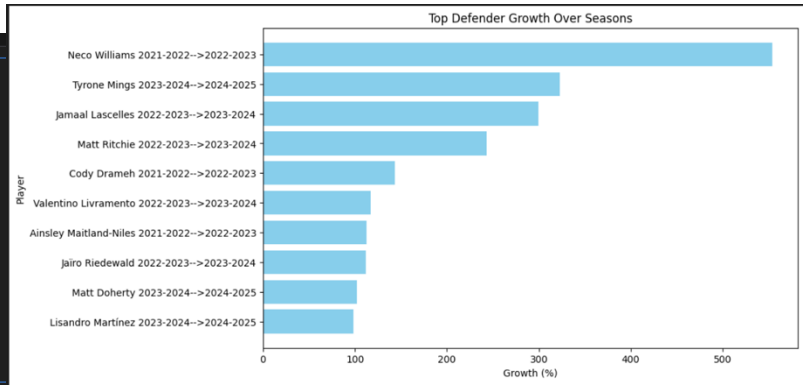
$$NormalisedAttributeScore_{ik} = Wj \cdot \frac{AVG(Attribute_{ijk})}{MAX(Attribute_j)}$$

These are summed to calculate two scores, one for the first season and one for the second. When score_1 > 0 (to avoid dividing by 0), growth is calculated with the following formula:

$$\frac{(RawPeformanceScore_1 - RawPeformanceScore_2)}{RawPeformanceScore_1} = Growth$$

Growth is then used to order data before returning the first ten defenders and the seasons in which they experienced the level of growth

	Player	Growth (%)	Seasons
0	Neco Williams	553.936818	2021-2022-->2022-2023
1	Tyrone Mings	323.102215	2023-2024-->2024-2025
2	Jamaal Lascelles	299.252562	2022-2023-->2023-2024
3	Matt Ritchie	243.040248	2022-2023-->2023-2024
4	Cody Drameh	143.478261	2021-2022-->2022-2023
5	Valentino Livramento	117.450442	2022-2023-->2023-2024
6	Ainsley Maitland-Niles	112.604352	2021-2022-->2022-2023
7	Jaïro Riedewald	111.791747	2022-2023-->2023-2024
8	Matt Doherty	101.780616	2023-2024-->2024-2025
9	Lisandro Martínez	98.736020	2023-2024-->2024-2025



Query 7: League Tables

PURPOSE: The purpose of this query is to present the league table for all the years stored within the database. This is to evaluate a teams performance in a given year.

CYPHER STEPS: First we get all matches for the selected season by filtering on the input boxes value. We then link the home and away team to the match so we know who played in it. As there can be matches in the future without scores, so we also filter where match is in the past. Now for every team that it links to, it assigns a score depending on if they won, lost or drew the game. This is via CASE. We then use unwind to break the match data into separate records for each team, treating home and away performances individually. This allows us to aggregate each teams' total goals scored, goals conceded and points over the season. Then we find the difference, per team, for goals scored and conceded and set this attribute to goals difference. The final league ranking is returned.

2021-2022 2022-2023 2023-2024 2024-2025

League Table: 2024-2025

Rank	Team Name	Points	Goal Difference
1	Liverpool	53	33
2	Arsenal	47	23
3	Nott'ham Forest	44	6
4	Manchester City	41	17
5	Newcastle Utd	41	14
6	Bournemouth	40	15
7	Chelsea	40	15
8	Aston Villa	37	-1
9	Brighton	34	4
10	Fulham	33	3
11	Brentford	31	2
12	Manchester Utd	29	-4
13	Crystal Palace	27	-4
14	West Ham	27	-16
15	Tottenham	24	9
16	Everton	23	-9
17	Leicester City	17	-24
18	Wolves	16	-20
19	Ipswich Town	16	-26
20	Southampton	6	-37

INPUT/OUPUT: To interact with this league table, it is done via the *'League Visualiser.ipynb'*. This first queries the database for all seasons stored and then presents these as buttons. After a season button is pressed it calls, the query and formats the response in a dataframe to show. The purpose of this query is to present the league table for all the years stored within the database. This is done by first, finding all the season nodes stored and outputting these as buttons for the user to press. To find all season nodes, we just use a match and return all. Once a league is selected, a query is executed which calculates how many points a team has acquired during the league, as well as their net goal difference (the amount of goals they've scored against how many they've conceded). This is used as a tie breaker for when two teams have the same amount of points in a league. This is then presented.

vi: Justification of Neo4j

Why Neo4j?

SCALABILITY: Neo4j's **structure** ensures that complex queries involving multiple relationships are executed faster than traditional SQL queries, which would require joining many tables. This is incredibly important as this database contains around 3.2k nodes and 50k relationships. Thus scalability of the database model is crucial.

RELATIONSHIPS ARE THE FOCUS: In football, the relationships between entities (the players, teams, matches, seasons) are incredibly important. Moreover, Neo4j excels in representing relationships between multiple entities, whereas relational databases are typically not optimimal for these complex relationships.

FLEXABILITY: We found neo4j was most suitable because in football, data models change over time (e.g., new stats, transfers, or rules). Therefore, the fact that Neo4j is **schema-less** is very important, as it allowed us to easily modify relationships.

Technical Restrictions Faced

One technical restriction that we initially encountered with Neo4J was being rate limited with connections to our instance. As we were inserting data individually, for each item we were adding we often encountered disconnects to the instance and sometimes got timed out. This method of inserting data was very inefficient and lead us to using unwind in our insertion queries, meaning that for every season we only need to execute 4 queries instead of thousands. Additionally another restriction faced was in the league ranking table. As cypher doesn't have an over() window function adding rankings in a league had to be done on the application side via pandas and not through the query.

Reproducibility Instructions

1. In terminal change directory to the main folder, then run the following command: $pip\ install - r\ requirements.txt$

```
Pillow-11.1.0 pandas-2.2.3
project-at2024-dam % pip install -r requirements.txt
```

2.Open Jupyter notebook named '*1. Environment Setup.ipynb'* and change the values within the '*to_insert*' dictionary to your own Neo4J Instance. Then run the cell, below to confirm the environment variables have been set correctly.

```
to_insert = {
    "NEO4J_URI":"neo4j+ssc://5fe6b298.databases.neo4j.io",
    "NEO4J_USERNAME": "neo4j",
    "NEO4J_PASSWORD":"V2S2unBr-ex60KUDHFzMH_KLkiPggGUR9UkEZYreAPM",
    "AURA_INSTANCEID":"5fe6b298",
    "AURA_INSTANCENAME":"Instance01"
}
```

3. (OPTIONAL) Regarding the seasons you would like to insert into the database, if they are not already within the data/season directory. Do the following: open the jupyter notebook named *'2. Webscraper.ipynb'* and find the list named premier_league_seasons and specify which seasons you would like to webscrape. This should be in the list in a string with yyyy-yyyy format. Then run the cell.

```
# declare the seasons that we wish to scrape for. Change if you wish
premier_league_seasons = ["2021-2022", "2022-2023", "2023-2024", "2024-2025"]
```

Note: This step will take at least 20 minutes per season due to rate limiting set by FBRef.

4. Now the final step to create and insert the data into a Neo4J instance is to open the jupyter notebook named *'3. Neo4J Data Insertion.ipynb'* and run the cell. You should see output from the script with the seasons it has inserted. You can now check via the Neo4J website, or other visualisers that nodes and relations have been inserted properly- there should be around 3.2k nodes and 50k relationships.

```
with driver.session(database="neo4j") as session:
    session.run(venue_capacity_query, parameters={"venues": venue_capacity_dict })
```

```
2021-2022
2022-2023
2023-2024
2024-2025
```